

A note on Is2EdgeTransitive

Frankie Gillis

October 2025

1 Introduction

Recall that a digraph is an ordered pair $\Gamma = (V, E)$, where V is the vertex set, and $E \subseteq V \times V$ is the edge set, which is a collection of ordered pairs of elements of V . The automorphism group $\text{Aut}(\Gamma)$ of Γ is the group of permutations of V which fix E setwise.

Definition 1.1. Let Γ be a digraph. A *2-edge* in Γ is a triple (u, v, w) of distinct vertices such that both $(u, v) \in E$ and $(v, w) \in E$. The set of all 2-edges in Γ is denoted $T(\Gamma)$.

An example of a 2-edge is given in [Figure 1](#), and examples of non-2-edges are given in [Figure 2](#).

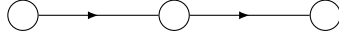


Figure 1: A 2-edge.

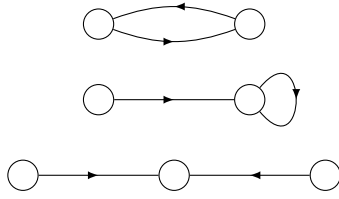


Figure 2: Not 2-edges.

Definition 1.2. Let Γ be a digraph. Then Γ is called *2-edge transitive* if the induced action of $\text{Aut}(\Gamma)$ on $T(\Gamma)$ is transitive. That is, given any pair of 2-edges (x, y, z) and (u, v, w) in Γ , there exists an automorphism φ of Γ such that

$$(\varphi(x), \varphi(y), \varphi(z)) = (u, v, w).$$

Definition 1.3. Let Γ be a digraph. A *2-cycle* in Γ is a triple (u, v, u) of vertices such that $(u, v), (v, u) \in E$. We say that (u, v, u) is a 2-cycle *based at* v .

2 Determining 2-edge transitivity

Determining 2-edge transitivity of a digraph Γ takes place in two steps: enumerate all 2-edges in Γ , then calculate the orbit length of a single 2-edge under $\text{Aut}(\Gamma)$. Then Γ is 2-edge transitive if and only if these numbers are equal. The orbit calculation is the most computationally-intensive step in the algorithm, and in general its time complexity is difficult to estimate. In practice, we calculate the stabiliser of a 2-edge, and then use the Orbit-Stabiliser theorem to find the orbit length.

Thus, we are left with the task of enumerating the 2-edges of Γ as efficiently as possible.

2.1 Enumerating 2-edges

Let Γ be a digraph with $|V(\Gamma)| = n$ and $|E(\Gamma)| = m$. Naively, the operation of computing the number of 2-edges in Γ has complexity $\mathcal{O}(n^3)$: we iterate over all n^3 elements of $V \times V \times V$ and increment the count every time an element satisfies the conditions to be a 2-edge. **GAP**-style pseudocode for this is given in Algorithm 1.

Algorithm 1 Naive implementation

```

1: procedure Is2EDGETRANSITIVE( $D$ )
2:    $O := \text{OutNeighbours}(D)$ ;
3:    $n := \text{Length}(O)$ ;
4:    $\text{twoEdges} := []$ ;
5:   for  $u$  in  $[1 .. n]$  do
6:     for  $v$  in  $O[u]$  do
7:       for  $w$  in  $O[v]$  do
8:         if  $u <> v$  and  $v <> w$  and  $w <> u$  then
9:            $\text{Add}(\text{twoEdges}, [u, v, w])$ ;
10:        end if
11:      end for
12:    end for
13:  end for
14:   $\text{numTwoEdges} := \text{Length}(\text{twoEdges})$ ;
15:  if  $\text{numTwoEdges} = 0$  then
16:    return true;
17:  else
18:     $G := \text{AutomorphismGroup}(D)$ ;
19:    return  $\text{Length}(\text{twoEdges}) * \text{Order}(\text{Stabiliser}(G, \text{twoEdges}[1])) = \text{Order}(G)$ ;
20:  end if
21: end procedure

```

In this section, we introduce a few definitions and lemmas which we will later use to significantly improve the performance of Algorithm 1.

Definition 2.1. Let $t = (u, v, w)$ be a 2-edge in a digraph Γ . The vertex v is called the *center* of t . The set of all 2-edge centers in Γ is denoted $C(\Gamma)$.

Lemma 2.2. *If Γ is 2-edge transitive, then $\text{Aut}(\Gamma)$ acts transitively on $C(\Gamma)$.*

Proof. Let $v, v' \in C(\Gamma)$. Then, there exist 2-edges $(u, v, w), (u', v', w')$ in Γ for some vertices $u, u', w, w' \in V(\Gamma)$. Since Γ is 2-edge transitive, there is an automorphism φ of Γ such that

$$(\varphi(u), \varphi(v), \varphi(w)) = (u', v', w').$$

In particular, $\varphi(v) = v'$. □

Unfortunately, the converse of [Lemma 2.2](#) is not true. Counterexamples to this are provided in [Example 2.3](#) and [Example 2.4](#).

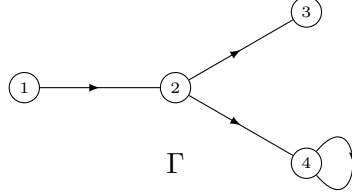


Figure 3: The digraph Γ .

Example 2.3. By inspection, we can see that $T(\Gamma) = \{(1, 2, 3), (1, 2, 4)\}$. Therefore $C(\Gamma) = \{2\}$ and so $\text{Aut}(\Gamma)$ clearly acts transitively on $C(\Gamma)$ via the identity automorphism. However, any automorphism mapping the 2-edge $(1, 2, 3)$ onto the 2-edge $(1, 2, 4)$ must map 3 onto 4, which is impossible as 3 has no loop while 4 has a loop. It follows that Γ is not 2-edge transitive.

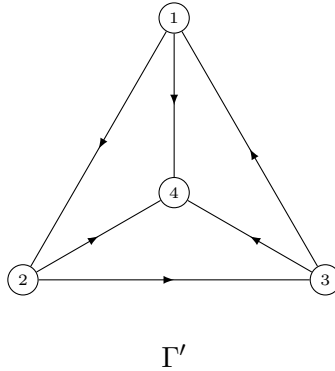


Figure 4: The digraph Γ' .

Example 2.4. First, note that $\text{Aut}(\Gamma') = \langle (1\ 2\ 3) \rangle$, the cyclic group of order 3 generated by the permutation $(1\ 2\ 3)$. By inspection, $C(\Gamma') = \{1, 2, 3\}$, and so $\text{Aut}(\Gamma')$ acts transitively on $C(\Gamma')$. However, note that there can be no automorphism of Γ' which maps the 2-edge $(1, 2, 3)$ onto the 2-edge $(1, 2, 4)$, as such a permutation must map the vertex 3 onto the vertex 4 which is impossible as $\deg_{\text{out}}(3) \neq \deg_{\text{out}}(4)$. Hence, Γ' is not 2-edge transitive.

If the converse of [Lemma 2.2](#) were true, we could compute 2-edge transitivity by determining whether $\text{Aut}(\Gamma)$ acts transitively on $C(\Gamma)$. This is advantageous as determining the latter has the complexity of computing vertex transitivity, which is much quicker than 2-edge transitivity. We do, however, get the following corollary which will become useful.

Corollary 2.5. *If Γ is 2-edge transitive, then for any $u, v \in C(\Gamma)$:*

1. $\deg_{in}(u) = \deg_{in}(v)$; and
2. $\deg_{out}(u) = \deg_{out}(v)$.

Proof. Follows from the fact that any automorphism of a digraph must preserve in-degree and out-degree. $\square$

Proposition 2.6. *Let Γ be a 2-edge transitive digraph. Then, there exist positive integers p, q and non-negative integers l, c such that for every $u \in C(\Gamma)$*

1. $\deg_{in}(u) = p$;
2. $\deg_{out}(u) = q$;
3. *There are l loops at u ; and*
4. *There are c 2-cycles at u .*

Proof. Let $u \in C(\Gamma)$ be any 2-edge center. Let $p = \deg_{in}(u)$, $q = \deg_{out}(u)$, let l be the number of loops at u and c be the number of 2-cycles at u .

Let $v \in C(\Gamma)$. By [Corollary 2.5](#), $\deg_{in}(v) = p$ and $\deg_{out}(v) = q$. Let φ be an automorphism of Γ such that $\varphi(u) = v$. Such an automorphism exists since $\text{Aut}(\Gamma)$ acts transitively on $C(\Gamma)$. The loop $(u, u) \in E(\Gamma)$ if and only if $(\varphi(u), \varphi(u)) = (v, v) \in E(\Gamma)$, so there are l loops based at v .

Similarly, for any $w \in V(\Gamma)$, the 2-cycle (w, u, w) exists in Γ if and only if $(w, u), (u, w) \in E(\Gamma)$. This happens if and only if $(\varphi(w), v), (v, \varphi(w)) \in E(\Gamma)$, so the 2-cycle (w, u, w) is in Γ if and only if $(\varphi(w), v, \varphi(w))$ is in Γ . Since there are c such 2-cycles at u , it follows that there are c 2-cycles at v .

$\square$

Proposition 2.7. *Let Γ be a 2-edge transitive digraph. Then,*

$$|T(\Gamma)| = [(p - l)(q - l) - c] |C(\Gamma)|.$$

Proof. First, we count the number of 2-edges in $T(\Gamma)$ whose center is v . Since $\deg_{in}(v) = p$, there are $p - l$ edges leading into v which are not loops. Similarly, there are $q - l$ edges leading out of v which are not loops. This gives $(p - l)(q - l)$ ways to choose an ordered triple (u, v, w) where neither (u, v) nor (v, w) are loops.

Now we must account for 2-cycles of the form (u, v, u) which have been counted. By assumption, there are c such 2-cycles, and these must be subtracted from the total. Therefore, there are

$$(p - l)(q - l) - c$$

2-edges with v as their center.

By the previous proposition, this is true for any $u \in C(\Gamma)$. It follows that

$$|T(\Gamma)| = [(p - l)(q - l) - c] |C(\Gamma)|.$$

$\square$

2.2 Determining l

Let l be the number of loops based at any $u \in C(\Gamma)$. Since we are not considering multidigraphs, $l \in \{0, 1\}$, as either $(u, u) \in E(\Gamma)$ or $(u, u) \notin E(\Gamma)$. Therefore, it suffices to check if $(u, u) \in E(\Gamma)$. This can be done in $\mathcal{O}(n)$ time by checking if u is an out-neighbour of u .

2.3 Determining c

Let c be the number of 2-cycles based at any $u \in C(\Gamma)$. That is,

$$c = \#\{(u, v, u) \mid (u, v), (v, u) \in E(\Gamma)\}$$

for any $u \in C(\Gamma)$. Equivalently,

$$c = \#(\{u \in V(\Gamma) \mid (u, v) \in E(\Gamma)\} \cap \{u \in V(\Gamma) \mid (v, u) \in E(\Gamma)\}).$$

This latter presentation is the most useful, as it says c is the size of the intersection of the in-neighbours of u and the out-neighbours of u . In **GAP**, the in-neighbours and the out-neighbours of a vertex are both ordered lists of integers, and we can compute the intersection size of two ordered lists of integers in $\mathcal{O}(n)$ time.

2.4 Determining centers

Let $v \in V(\Gamma)$. Set $p_0 = \deg_{\text{in}}(v)$, $q_0 = \deg_{\text{out}}(v)$, l_0 as the number of loops at v , and c_0 as the number of 2-cycles at v . By [Proposition 2.7](#), the number of 2-edges whose center is v is

$$(p_0 - l_0)(q_0 - l_0) - c_0.$$

In particular, $v \in C(\Gamma)$ if and only if

$$(p_0 - l_0)(q_0 - l_0) - c_0 > 0.$$

3 The Algorithm

3.1 Analysis of Time Complexity

Enumeration of 2-edges in [Algorithm 2](#) has time complexity $\mathcal{O}(n^2 + m)$. The m term comes from building the InNeighbours list, as digraphs in **GAP** are stored by OutNeighbours. The n^2 term comes from looping through each vertex, which is $\mathcal{O}(n)$ time, and within each loop calculating l and c , each of which take $\mathcal{O}(n)$ time, giving $\mathcal{O}(n^2)$ time in total. This certainly beats the $\mathcal{O}(n^3)$ of [Algorithm 1](#), since $m \leq n^2$.

Algorithm 2 Faster implementation

```
1: procedure Is2EDGETRANSITIVE( $D$ )
2:    $O := \text{OutNeighbours}(D)$ ;
3:    $I := \text{InNeighbours}(D)$ ;
4:    $n := \text{Length}(O)$ ;
5:    $p := 0$ ;
6:    $q := 0$ ;
7:    $l := 0$ ;
8:    $c := 0$ ;
9:    $\text{centers} := []$ ;
10:  for  $u$  in  $[1 .. n]$  do
11:     $p_0 := \text{Length}(I[u])$ ;
12:     $q_0 := \text{Length}(O[u])$ ;
13:    if  $u$  in  $O[u]$  then
14:       $l_0 := 1$ ;
15:    else
16:       $l_0 := 0$ ;
17:    end if
18:     $c_0 := \text{IntersectionSize}(I[u], O[u])$ ;
19:    if  $(p_0 - l_0) * (q_0 - l_0) - c_0 > 0$  then
20:       $\text{Add}(\text{centers}, u)$ ;
21:      if  $p = 0$  then
22:         $p := p_0$ ;
23:         $q := q_0$ ;
24:         $l := l_0$ ;
25:         $c := c_0$ ;
26:      else
27:        if  $p <> p_0$  or  $q <> q_0$  or  $l <> l_0$  or  $c <> c_0$  then
28:          return false;
29:        end if
30:      end if
31:    end if
32:  end for
33:   $\text{numTwoEdges} := ((p - l) * (q - l) - c) * \text{Length}(\text{centers})$ ;
34:  if  $\text{numTwoEdges} = 0$  then
35:    return true;
36:  else
37:     $v := \text{centers}[1]$ ;
38:    for  $u$  in  $I[v]$  do
39:      for  $w$  in  $O[v]$  do
40:        if  $u <> v$  and  $v <> w$  and  $w <> u$  then
41:           $\text{twoEdge} := [u, v, w]$ ;
42:           $G := \text{AutomorphismGroup}(D)$ ;
43:          return  $\text{numTwoEdges} * \text{Order}(\text{Stabiliser}(G, \text{twoEdge})) = \text{Order}(G)$ ;
44:        end if
45:      end for
46:    end for
47:  end if
48: end procedure
```

4 Plots

4.1 Algorithm 1 versus 2

